GABRIEL POPA

Senior Software Engineer - Python(Django/FastAPI) • AWS
Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | https://www.linkedin.com/in/gabriel-popa-12890b414/

ABOUT ME

Senior Software Engineer with **10+ years** of experience building reliable backend platforms for **web product, operations, and data-driven business workflows**, with strongest focus on **Python backend engineering, API design, asynchronous processing, database performance, and cloud deployment** across fast-moving product teams. Strong hands-on depth with **Python 2.7–3.12, Django 1.x–4.x, FastAPI 0.6+–0.110+, Flask 0.10+–3.x, PostgreSQL, Redis, Celery, RabbitMQ, Docker, and AWS**, with a practical track record of fixing production failures, modernizing legacy services, stabilizing releases, improving observability, and delivering maintainable backend features that balance speed, correctness, and long-term system health.

SKILLS

- **Backend: Python 2.7–3.12**, **Django 1.x–4.x**, **FastAPI 0.6+–0.110+**, **Flask 0.10+–3.x**, REST API design, authentication, authorization, background jobs, validation, error handling, service integration, batch processing, ETL support
- **Data & Messaging: PostgreSQL**, MySQL, SQLite, **Redis**, **Celery**, **RabbitMQ**, query optimization, caching, reporting workflows, scheduled jobs, transaction handling, concurrency control
- **Cloud & DevOps: AWS**, Docker, Linux, Nginx, CI/CD, GitHub Actions, Jenkins, deployment automation, environment configuration, monitoring support
- **Observability / Quality / Security:** structured logging, **OpenTelemetry basics**, **Prometheus / Grafana basics**, **Sentry**, rate limiting, OWASP security practices, **Pytest**, unit testing, integration testing, API testing, contract testing, end-to-end testing, error monitoring

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania *(Jan 2024 – May 2026)*

- Led modernization of a backend platform using **Python 3.11–3.12** and **FastAPI**, restructuring older service logic into clearer API and domain boundaries that improved maintainability and reduced delivery friction on workflow-heavy modules.
- Owned a recurring failure pattern where invalid payloads triggered inconsistent responses across endpoints, then introduced stricter request validation, unified exception handling, and safer response contracts that reduced integration defects by **32%**.
- Improved reporting performance by profiling expensive **PostgreSQL** queries, reducing unnecessary joins, and introducing selective **Redis** caching, which stabilized response times on filter-heavy dashboards and improved daily operational usage.
- Resolved a production issue where concurrent edits occasionally overwrote recently saved values by implementing optimistic concurrency checks and clearer conflict handling, which reduced silent data-loss incidents by **28%**.
- Moved long-running export and synchronization operations out of synchronous request paths into **Celery** workers backed by **Redis**, which reduced timeout-related failures by **37%** and improved user confidence in large jobs.
- Investigated memory growth in a backend batch-processing flow by tracing object retention and processing patterns, then split the job into smaller units with safer cleanup logic that improved worker stability under heavy datasets.
- Introduced structured logging, **Sentry**, and basic **OpenTelemetry** instrumentation across key request paths, which shortened root-cause analysis time by **35%** during production incident investigation.
- Closed security gaps on sensitive API routes by tightening authentication checks, improving validation rules, and applying practical rate limiting, which lowered abuse exposure while keeping normal user flows stable.

- Expanded automated coverage with **Pytest** and API-focused regression checks around validation, permissions, and background processing, which reduced backend release regressions by **30%** during frequent feature delivery cycles.
- Supported a staged runtime and dependency upgrade for older Python services, using targeted refactors and stronger test coverage to move the application onto a more supportable baseline without destabilizing active modules.
- Improved deployment predictability by tightening **GitHub Actions** validation and release checks, which reduced environment-specific failures by **24%** and made production rollouts more repeatable for the team.
- Worked closely with product, QA, and engineering stakeholders to break unclear backend requirements into smaller safe releases, helping the team deliver new service behavior without sacrificing stability or observability.

Software Engineer

Next Apps | Ghent, Belgium *(Nov2020 – Nov 2023)*

- Delivered backend product features with **Python 3.8–3.11**, **Django**, and selected **FastAPI** services, helping unify mixed implementation styles into cleaner and more maintainable service behavior across the platform.
- Stabilized fragile third-party integrations by adding retry handling, payload normalization, defensive parsing, and clearer logging, which reduced user-facing sync failures by **29%** when upstream systems behaved inconsistently.
- Solved stale-data issues in reporting flows by improving **Redis** invalidation logic and backend update sequencing, which reduced cache inconsistency problems by **26%** and improved trust in operational totals.
- Strengthened API reliability by standardizing validation and response patterns across older and newer endpoints, which reduced ambiguity for consuming clients and lowered downstream error-handling complexity.
- Resolved duplicate-processing and stuck-task issues in asynchronous workflows by reviewing **Celery** retry policies, timeout handling, and cleanup behavior, which improved job completion reliability by **31%**.
- Improved production visibility by adding **Sentry** and more useful structured logging around backend failures, allowing the team to identify and prioritize real incidents **40%** faster before escalation.
- Helped standardize local and deployment environments with **Docker** and cleaner configuration practices, which reduced backend onboarding friction and made production-like defects easier to reproduce in development.
- Introduced stronger **Pytest** coverage around permissions, validation logic, and service-layer changes, which improved release confidence and reduced avoidable backend regressions by **27%**.
- Tightened exposed endpoint security through stronger request validation, authentication checks, and rate limiting, which improved resilience against malformed traffic without disrupting normal system usage.
- Supported incremental service refactoring in modules that mixed business rules and transport logic, producing cleaner separation that made new backend features easier to add and safer to review.
- Worked across feature delivery and defect resolution in a fast-moving environment, using Python tooling flexibly to balance new implementation work with stability improvements and operational support.

Software Developer

Datamart | Stockholm, Sweden *(Feb 2016 – Sep 2020)*

- Worked on internal business applications with **Python 3.5–3.8**, **Django**, and **Flask**, helping evolve reporting and operational workflows into more maintainable backend services without forcing disruptive architecture shifts.
- Supported backend improvements for search, approval, and export flows where unstable behavior slowed day-to-day operations, delivering targeted fixes that improved reliability for internal users by **22%**.
- Solved a recurring reporting defect caused by timezone mismatches between user input and backend processing, then normalized parsing and storage rules to improve consistency in scheduled and exported data.
- Improved slow administrative screens by rewriting inefficient **PostgreSQL** queries, reducing repeated reads, and adding missing indexes, which reduced backend database load by **35%** on high-use reporting paths.
- Built and maintained internal REST-style endpoints where backend and presentation concerns had started to overlap, then improved contract clarity and responsibility boundaries to support safer incremental changes.
- Reduced operational strain during heavy imports by moving long-running backend work into scheduled and asynchronous processing paths, which improved request stability by **25%** during high-volume administrative activity.
- Improved deployment safety by supporting **Jenkins** jobs, environment validation, and release scripts, which lowered packaging and configuration mistakes by **30%** in a mostly manual release process.
- Made backend defects easier to troubleshoot by improving logging quality and surfacing clearer failure information in reporting and batch workflows, reducing investigation time for vague production issues.

- Added focused **Pytest** coverage around validation, transformation, and scheduled-processing logic, which reduced regression risk and made routine maintenance safer for older backend modules.
- Worked within historically practical stack choices for the period, using Python, Django, Flask, SQL, and background processing patterns in ways that matched platform maturity and minimized unnecessary migration risk.

Junior Software Developer

Berg Software | Timișoara, Romania *(Sep2014 – Mar 2016)*

- Contributed to a legacy business application built with **Python 2.7–3.5** and **Django**, helping deliver smaller backend changes in tightly coupled modules where stability mattered more than large-scale redesign.
- Maintained backend forms, validation rules, and request-handling logic in server-rendered workflows, keeping daily user operations reliable while senior engineers handled broader architectural decisions.
- Fixed a recurring login and session-stability issue by helping align cookie behavior, timeout handling, and request flow assumptions, which improved sign-in reliability by **20%** for regular users.
- Improved form-processing behavior in several admin areas by tightening backend validation and correcting edge-case input handling, which reduced failed submissions by **25%** and made error recovery clearer.
- Assisted with defect investigation by reproducing issues from logs, tester notes, and support feedback, then preparing focused backend fixes that were easier to review and safer to release.
- Helped improve slower administrative pages by reducing unnecessary database queries and simplifying backend data preparation, which improved response behavior without introducing risky structural change.
- Supported framework and dependency update work by checking compatibility issues and correcting deprecated backend code paths, which reduced upgrade-related regressions in fragile application areas.
- Built small helper utilities for repeated backend behaviors across forms and administration flows, lowering copy-paste logic and making later maintenance changes more consistent for the team.
- Added basic tests and extra logging in selected backend workflows while assisting QA and release validation, which helped the team catch common defects earlier and improve support-side debugging.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (*2010 – 2014)*

- Focused on practical software engineering fundamentals that translated directly into building reliable web systems and data-driven applications.

CERITIFICATIONS

AWS Certified Developer – Associate — 2023

- Demonstrated practical capability in developing and maintaining **AWS**-based applications, with focus on deployment, service integration, monitoring, and secure cloud operations.

PCAP – Certified Associate in Python Programming — 2022

- Demonstrated strong foundation in **Python** programming, including core language features, modular design, debugging, and problem-solving for backend-oriented applications.

Scrum Fundamentals Certified (SFC) — 2021

- Demonstrated working knowledge of agile principles, sprint-based execution, team collaboration, and structured delivery practices within software development environments.